

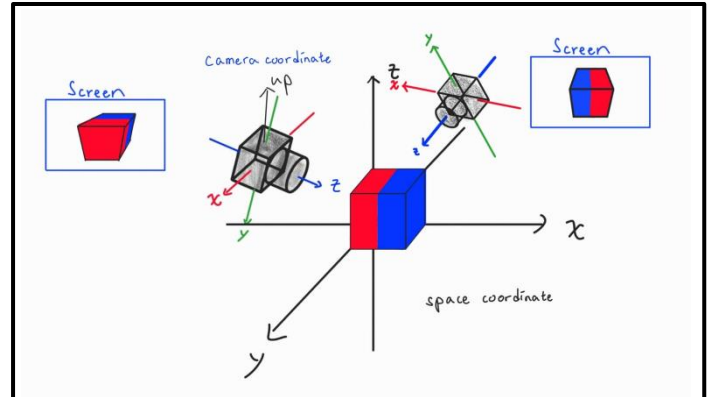
三維模型渲染加速器

110060021 曾偉博

110060035 黃振寧

一、簡介

本專案旨在設計並實現一個三維模型渲染加速器(如圖一)，針對不同的相機的座標位置、相機的面相方向，以及標示相機上方的向量，針對不同三維模型渲染。三圍模型的資訊將會存在 sram 中，我們支援的三維模型格式是三角網格的 PLV，會提前將欲渲染的模型存入，如每一個面的頂點編號以及每一個頂點的頂點的空間座標 (x, y, z) 與顏色資訊 (r, g, b)，。我們採用的坐標系是與 OpenGL 相同的右手坐標系。



(圖一)

針對 **Vertex Shader** 與 **Rasterizer** 兩個模組，我們設定了不同的設計目標。光柵化的處理速度是渲染過程中的主要瓶頸，因此，我們希望 **Rasterizer** 能夠以盡可能快的速度完成其任務，而 vertex shader 則是在 rasterizer 光柵化結束前轉換完座標，並且面積越小

二、演算法

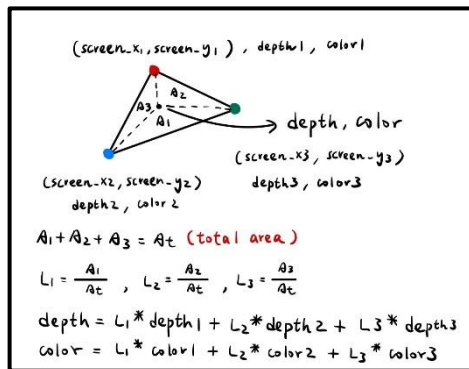
Vertex shader

此模組的主要任務是執行頂點轉換，將物體頂點從模型的世界座標系統轉換至螢幕座標系統。我們透過一連串的矩陣運算將物體變換至不同空間座標系(1.World space 2.View space 3.Projection space 4.Clip space 5.Normalized device space 6.Screen space)，我們採用的是視錐投影而非正交投影以達到更好的效果。

在進行轉換之前，我們須先計算相機的座標系以及其對應的 View matrix，會依序計算以下資訊：1. Z 軸：相機位置減去相機螢幕中心位置並將其正規化 2. X 軸：將相機的上向量外積 Z 軸並將其正規化 3. Y 軸：透過 Z 軸與 X 軸外積(由於兩者皆會是單位向量且正交，因此 Y 軸計算結果也會是單位向量)4. 計算 view matrix(如下圖) 5. 計算 MVP：model matrix 乘以 view matrix 乘以 projection matrix。

完成以上步驟後我們便可以開始轉換頂點，對於頂點則需透過以下幾個步驟：1. 將輸入的向量延伸成為齊次座標 2. 將齊次座標與 MVP matrix 相乘變換到 clip space 3. 將頂點的齊次座標進行齊次除法，以獲得 $[-1, 1]$ 範圍內的 NDC 座標 4. 使用 View port transformation，將頂點轉換到 Screen space。

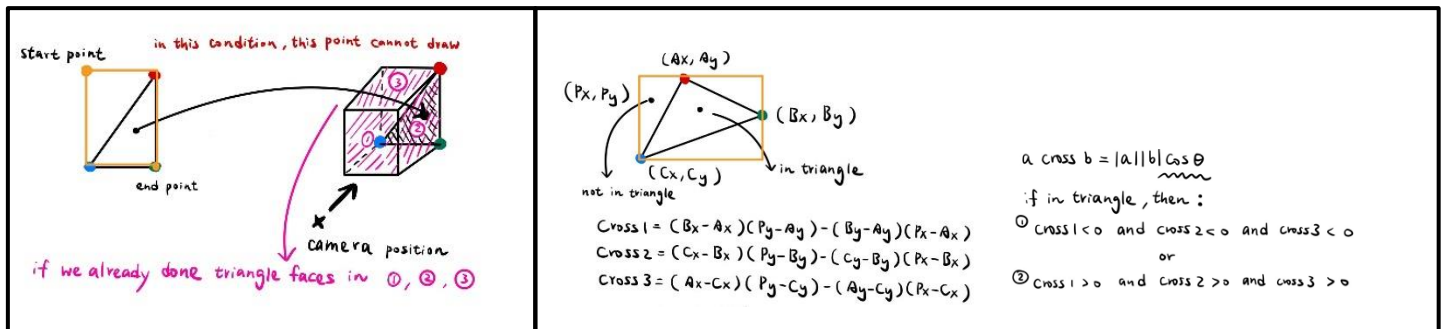
Rasterizer



(圖二)

此模組的功能是將三角形平面進行渲染，並且透過面積差值法(圖二)求得三角形平面中所有點的深度和 RGB 顏色，深度指的是組成 3D 物體的三角形網格平面投影到 2D 螢幕時的前後關係，透過這種前後關係，我們就可以決定當多個平面在投影到 2D 螢幕有重疊時，應該要畫哪些點在螢幕之上(圖三)。除此之外，為了遍歷三角形網格平面，我們會先尋找包含三角形網格的最小矩形，並從矩形的左上方開始遍歷整個矩形，不過由於這種方法會遍歷到一些不在該三角形網格的 pixel，因此我們會透過內積法判斷某一點是否在三角形內部

(圖四)。最後，我們還會判斷這個三角形網格是否為一個合法的面，假如出現任兩頂點重疊的狀況，我們就會判斷這不是一個合法的面，因此不會去做渲染的動作。

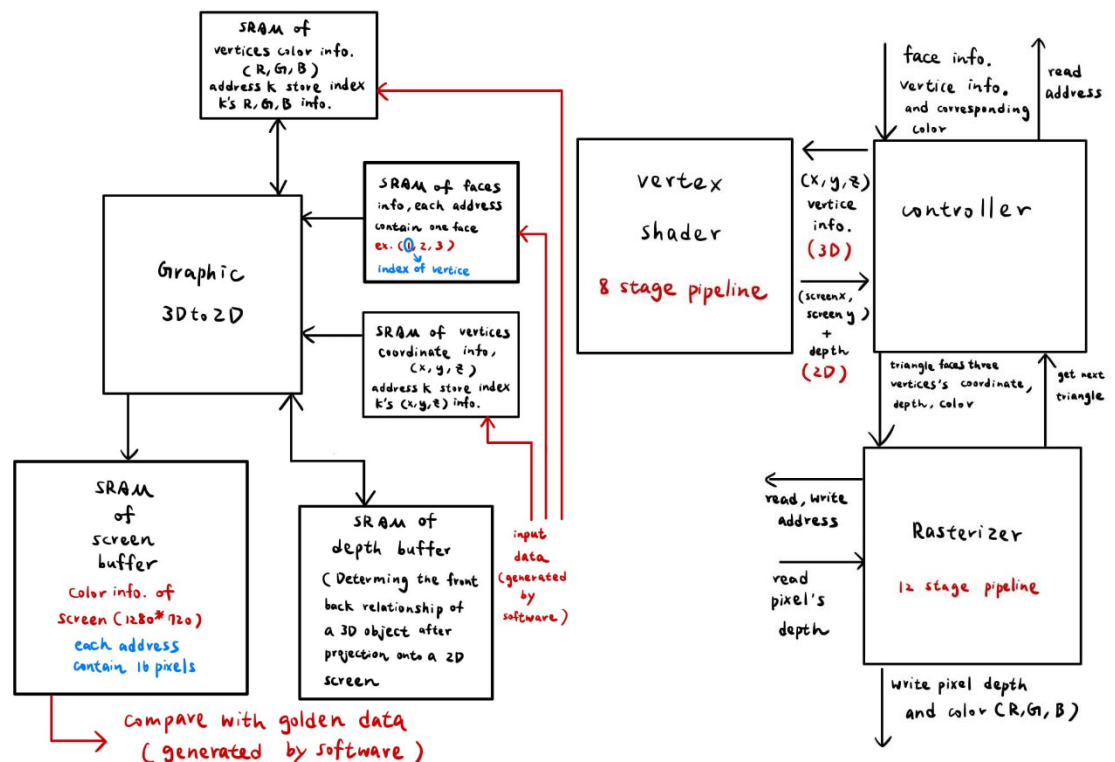


(圖三)

(圖四)

三、硬體實作

Top module 及 Sram 配置

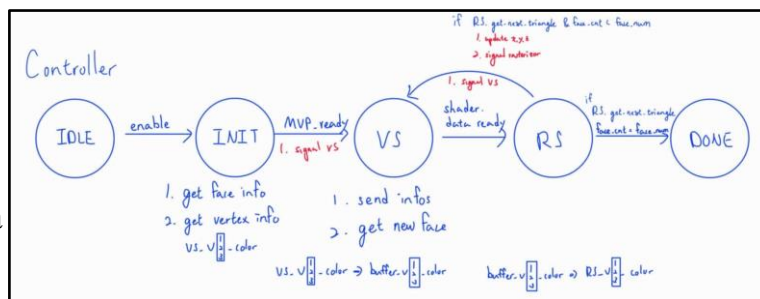


1. Controller

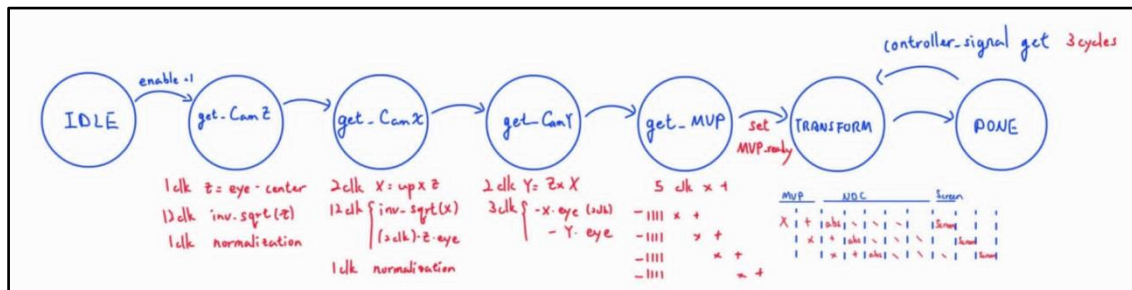
此模組的存在目的有二，

一是為了溝通 Rasterizer 及 vertex shader，二是為了欲先將 Sram 中 data 讀入 register 以節省等待時間

2. Vertex shader



(圖五)



(圖六)

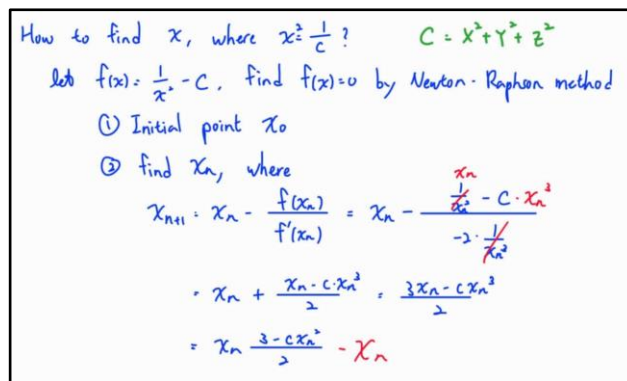
以上每一個 state 中的運算皆會切 pipeline, critical path 為 24bit*24bit 乘法。此處的難點在於正規化以及計算需足夠精準卻又希望面積越小越好的狀況，為了提高精準度以及提高 resource sharing，我們的所有乘法皆是使用 24bit 實做，並透過不斷改變定點數的 fl，達到高精準度的運算，舉例來說，我們的內積、外積以及尋找平方根倒數的模組都是支援 4Q20 的運算，然而，我們的需求不只有 4Q20，還有 1Q23，我們可以透過給值前後乘以一定倍數來達到維持精度且共用模組的效果。

正規化實作方式

因為要正規化的 input 為 25bit，並且運算精準度須達到 2^{-20} ，因此我們透過 Newton-Raphson method 來實現 $\frac{1}{|v^2|}$ （先查表後泰勒展開逼近，疊代三次），推倒過程如右圖，我們目標是尋找 $f(x) = 0$ 時的 x ，因此，我們在每次疊代時，我們將尋找切線與 x 軸的。此模組所接收的 (x, y, z) 大於 1 且為 4Q20 的定點數，因此，我們透過整數位查表得初始點，並且由於初始點足夠精準，我們可以保證在迭代過程中，所有疊代值將小於 1，因此在運算過程中，我們不僅會 quantize LSB，也會 truncate。為了滿足上述的要求，我們在正規化 X 軸時，會把 X 以及 Z 為單位向量，因此 X 必為小於等於 1 的向量判斷）。

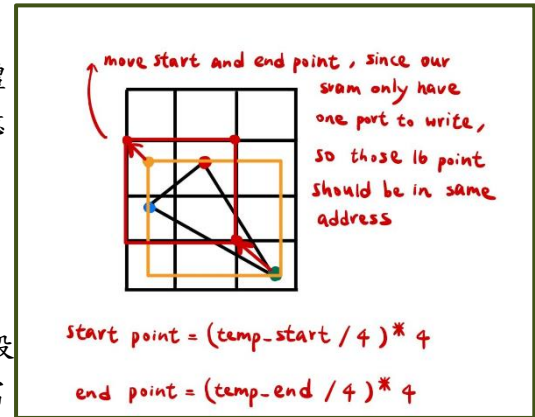
3. Rasterizer

我們在 Rasterizer 採用 12-stage pipeline，並且同時平行化計算 16 個在三角形網格內的像素，之所以平行化的對象選擇在同一三角形網格平面內的像素而不是同時做多個不同的三角形網



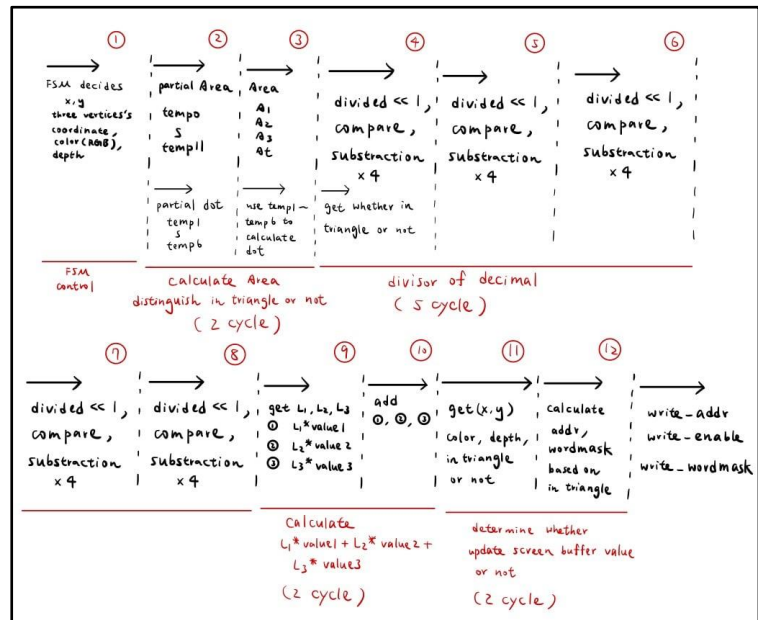
(圖七)

格平面，則是考慮到不同三角形網格平面投影後可能會重疊，產生 hazard 的問題，並且此種作法使用的記憶體需要支援同時寫入 16 個不同 address 的寫入，這種記憶體我們在之前並沒有使用過，因此不確定本次專案是否允許，所以最終選擇對同一三角形網格內的像素實現平行化計算，不過，由於我們在同一個三角形網格內部同時處理 16 個像素，因此我們的一個 address 會存 16 個 pixel，共 384 bits ($3 \times 8 \times 16$)，並且由於我們的 sram 設計成同時只有一個 port 可以寫入，因此我們還需要對需要遍歷的矩陣做出調整(圖八)。至於 pipeline 的部分，



(圖八)

由於 3D 物體與 camera 的距離會大幅影響 vertex shading 過後三角形平面的大小(越近越大)，因此我們希望透過 pipeline 的方式讓轉換後三角形網格平面的大小不會是影響我們運算所花費 cycle 數的主因。除此之外，越不精緻的 3D 模型通常在同樣的尺寸大小下，一個三角形網格平面的面積會比較大，我們同樣希望越不複雜的模型可以運算更快，因為複雜的 3D 建模通常在場景中不會很常出現，假如不使用 pipeline，除了運算複雜模型的速度會變慢之外，就連計算簡單的模型也會花非常多的 cycle。最後，選擇 12-stage 的原因則是在於 controller

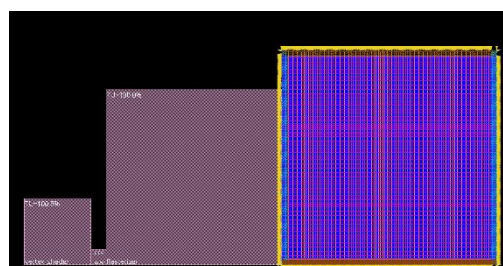


(圖九)

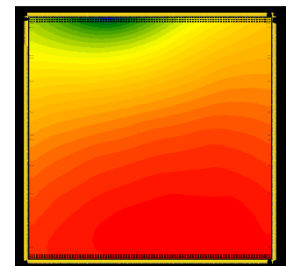
透過 sram 拿到某個三角形網格頂點的座標至少需要 8 個 cycle (get face' s three index: 3 cycle, get three vertex coordinate: 5 cycle)，因此至少需要 8 級的 pipeline 才會比較划算，因為假如小於 8 級 pipeline，在三角形網格平面很小的狀況下(模型距離較遠或是 3D 模型較精緻)，算完後無法立刻進行下一次的三角形平面計算，因此我們會讓 pipeline 切的級數大於 8。然而 pipeline 切太多級卻又會導致面積急速上升，在思考過後，我們決定以 26 bit 乘法作為 critical path，因此最終決定切了 12 級的 pipeline(圖九)。

四、SYN 以及 APR

我們合成的 timing 為 8.0ns，合成完面積為 $1359960 \mu\text{m}^2$ ，由於許多資料皆是存於 sram，因此我們設定的 input/output external delay 為 50% timing。



(圖十)



(圖十一)

APR 面積為 $1600084\mu\text{m}^2$ ，utilization 為 85%(1359960/1600084)，timing 為 10.0ns。chip layout 以及 IR drop 結果請見(圖十)以及(圖十一)。Power 分析如下表

	Pre layout	Post Layout (pre-sim waveform)	Post layout (post-sim waveform)
Net Switching Power	$7.244 \times 10^{-3} \text{ W}$	0.0115 W	0.0174 W
Cell Internal Power	0.0372 W	0.0228 W	0.0365 W
Total Power	0.0445 W	0.0343 W	0.0539 W

五、成果

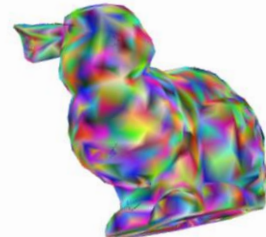
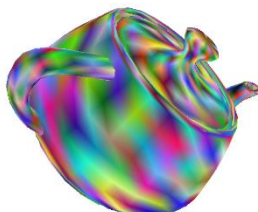
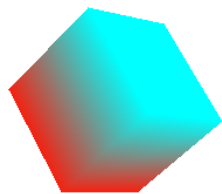
本次 testcase 中最多面以及頂點的測資為 Stanford bunny，一共有 2915 個面，以及 1493 個頂點，而我們渲染一張含有 Stanford Bunny 的圖片需要 $148800(\text{cycle}) * 10\text{ns} (\text{timing}) = 0.001488\text{s} > 672 \text{ fps}$ 。由於渲染時間與渲染面數呈正相關，根據我們估算，我們支援 144FPS 渲染有著約一萬面與頂點的三維模型，可以說是符合預期，至於沒有拿上萬點與面的模型來模擬的原因是，模擬時間會過久，目前約 3000 個點的 gatesim + postsim 便要 4 小時才會執行結束。

以下附上我們測資的渲染結果
不同角度下的 Dragon



下面為其餘五種 3D 模型渲染結果

依序為 elephant、box、Utah' s teapot、Athena、Stanford bunny



註

在此次專案中所有檔案，包括軟體(.ipynb)、硬體設計(.v)以及 3D 網格(.ply(上網下載後著色))接是為這個課程設計，在這之前沒有任何的資料，並且完全是沒有引用開源程式碼。